



DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

MSR INSIGHT - SMART ANALYTICS

submitted to

RAMAIAH INSTITUTE OF TECHNOLOGY

Bengaluru

MINI PROJECT (ISP67)

Submitted by

KARTHIK S POOJARY

1MS23IS051

AJAY KUMAR S

1MS24IS400

AKASH SHIVANAND NAVANI

1MS24IS402

AKSHAY KUMAR

1MS24IS403

Bachelor of Engineering in Information Science & Engineering

Under the guidance of

PRATHIMA M. N.

Assistant Professor

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)



DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

A Dissertation Report on

MSR INSIGHT - SMART ANALYTICS

Submitted by

KARTHIK S POOJARY	1MS23IS051
AJAY KUMAR S	1MS24IS400
AKASH SHIVANAND NAVANI	1MS24IS402
AKSHAY KUMAR	1MS24IS403

Bachelor of Engineering in Information Science & Engineering

Under the guidance of

PRATHIMA M. N.

Assistant Professor

Information Science & Engineering

RAMAIAH INSTITUTE OF TECHNOLOGY

Ramaiah Institute of Technology

(Autonomous Institute, Affiliated to VTU)

BANGALORE-560054

Department of Information Science & Engineering



CERTIFICATE

This is to certify that the project work titled “MSR Insight - Smart Analytics” is a bonafide work carried out by **Karthik S Poojary (1MS23IS051), Ajay Kumar S (1MS24IS400), Akash Shivanand Navani (1MS24IS402), and Akshay Kumar (1MS24IS403)** in partial fulfillment for the course of Bachelor of Engineering in Information Science and Engineering during the year 2026. The project report has been approved as it satisfies the academic requirements with respect to the project work. To the best of our understanding, the work submitted in this report has not been submitted anywhere else for the award of any degree or diploma.

Signature of the Guide

Signature of the HOD

External Examiners

Name of the Examiners:

Signature

- 1.
- 2.

DECLARATION

We hereby declare that the entire work embodied in this “**MSR Insight - Smart Analytics**” report has been carried out by us at Ramaiah Institute of Technology under the supervision of **PRATHIMA M. N.** This project report has not been submitted in part or full for the award of any diploma or degree of this or any other University.

Place: Bangalore

Date:

Karthik S Poojary	1MS23IS051
Ajay Kumar S	1MS24IS400
Akash Shivanand Navani	1MS24IS402
Akshay Kumar	1MS24IS403

ACKNOWLEDGEMENT

First and foremost, my utmost gratitude to PRATHIMA M. N, dept of ISE, MSRIT whose sincerity and encouragement we will never forget. She has been our inspiration as we overcame all the obstacles in the completion of this project work.

Dr. Sumana M, Head of the Department of Information Science and Engineering, had kind concerns and consideration regarding project work and we would like to thank her for continued support.

We would like to thank our beloved principal Dr. N.V.R Naidu for his support and encouragement.

This work would not have been possible without the guidance and help of several individuals who in one way or another contributed their valuable assistance in preparation and completion of this study.

We would like to express sincere thanks to all the teaching and non-teaching faculty of ISE Department, our beloved parents and my dear friends who helped in all the ways while preparing the Report.

ABSTRACT

The MSR Insight - Smart Analytics is an industry-grade, AI-powered academic reporting platform developed to automate the generation of professional and insight-driven student performance reports. In the modern educational environment, managing and analysing academic data manually is a time-consuming process that is often prone to human errors and inconsistent feedback. It addresses these challenges by introducing an intelligent reporting system capable of converting raw student data into meaningful academic insights. The platform is built on a scalable three-tier architecture consisting of a Next.js-based frontend for user interaction, an Express.js backend for workflow management, data scraping, and secure database communication, and a FastAPI-powered intelligence service for AI-driven analytics, vector database operations, and Retrieval-Augmented Generation (RAG).

The implementation of the system focuses on integrating advanced Generative AI models such as Llama 3.1 and Gemini, combined with robust RAG workflows, to improve the quality and accuracy of academic reporting. The platform allows proctors to query student records using natural language and generates personalized, context-aware remarks for student reports. PostgreSQL with pgvector is used for reliable data storage and semantic search capabilities, while Redis-based session management ensures secure and efficient workflow handling. The roadmap of the project includes modules for automated data scraping via Puppeteer, LLM-based report generation, semantic querying, and an interactive client-side grade simulator, all designed to provide a seamless and intelligent academic monitoring system.

The RAG-powered chatbot enables proctors to instantly retrieve relevant student data, helping educators take proactive measures for academic improvement. The system successfully bridges the gap between raw educational data and actionable academic strategies by delivering automated, intelligent, and user-friendly reports. In conclusion, the project provides an efficient and scalable solution for modern educational institutions, enhancing decision-making processes and supporting better academic outcomes for both educators and students.

CONTENTS

Declaration	iv
Acknowledgements	v
Abstract	vi
List of Figures	ix
List of Tables	ix
List Of Acronyms.....	x
1 INTRODUCTION	1
1.1 General Introduction	1
1.2 Problem Statement	2
1.3 Objectives of the Project	2
1.4 Project Deliverable	2
1.4.1 Deliverable Development and ReviewProcess.....	2
1.5 Current Scope	3
2 PROJECT ORGANIZATION	4
2.1 Software Process Models	4
2.2 Roles and Responsibilities	4
2.3 Finance–Budget Estimation	5
2.4 Sustainable Development Goals (SDGs)	6
3 LITERATURE SURVEY	8
3.1 Related Works with References	8
3.2 Conclusion of Survey	17
4 PROJECT MANAGEMENT PLAN	18
4.1 Schedule of the Project	18
4.2 Risk Identification	20
5 SOFTWARE REQUIREMENT SPECIFICATIONS	21
5.1 Functional Requirements	21
5.2 Non-Functional Requirements	23

6 DESIGN AND IMPLEMENTATION	24
6.1 Architecture Design	24
6.2 Data Flow Diagram	25
6.3 Tools and Technology Introduction	26
6.4 Overall Implementation View	26
6.5 RAG and Generative AI Implementation.....	28
6.5.1 Vectorizing Student Data using langchain_postgres.....	28
6.5.2 Setting up Embeddings using langchain_google_genai.....	28
6.5.3 Retrieval Mechanisms inside rag_service.py.....	28
6.6 Implementation of Modules	29
6.7 Conclusion	31
7 RESULT AND DISCUSSION	32
7.1 Experimental Setup	32
7.2 Evaluation Metrics	32
7.3 Results	33
7.4 Comparative Analysis	34
7.5 Discussion	35
8 TESTING	36
8.1 Testing Tools and Environment	36
8.2 Test Cases	36
9 CONCLUSION & SCOPE FOR FUTURE WORK	37
10 REFERENCES	38

LIST OF FIGURES

Figure 4.1 Project Schedule Gantt Chart	18
Figure 6.1 Application Data Flow Diagram	25
Figure 6.2 Login Authentication Interface	27
Figure 6.3 Landing Page Interface	27
Figure 6.4 Student Login Interface	29
Figure 6.5 Proctor Login Interface	30
Figure 7.1 Generated AI PDF Report Output	34
Figure 7.2 Comparative Latency Analysis Graph	35

LIST OF TABLES

Table 2.1 Project Roles and Responsibilities	5
Table 2.2: Finance and Budget Estimation.....	6
Table 4.1 Risk Identification and Mitigation Matrix	20
Table 5.1 Software Hardware & Environmental Requirements	21
Table 6.1 Database Schema Definitions	31
Table 7.1 RAG and LLM Latency Benchmarks	33
Table 8.1: Functional Test Cases Summary.....	36

LIST OF ACRONYMS

ACID: Atomicity, Consistency, Isolation, Durability

API: Application Programming Interface

CORS: Cross-Origin Resource Sharing

CTA: Call to Action

DOM: Document Object Model

HTML: HyperText Markup Language

JSON: JavaScript Object Notation

JWT: JSON Web Token

LLM: Large Language Model

LMS: Learning Management System

LTI: Learning Tools Interoperability

LSTM: Long Short-Term Memory

ORM: Object-Relational Mapping

RAG: Retrieval-Augmented Generation

REST: Representational State Transfer

RMSE: Root Mean Square Error

RNN: Recurrent Neural Network

SDK: Software Development Kit

SOA: Service-Oriented Architecture

SQL: Structured Query Language

TTL: Time-To-Live

UI: User Interface

USN: University Seat Number

UUID: Universally Unique Identifier

UX: User Experience

Chapter 1

INTRODUCTION

This project covers the development of **MSR Insight – Smart Analytics**, an AI-powered academic analytics and reporting platform. It automates student data collection, analysis, and report generation to improve academic evaluation. The system uses technologies like Machine Learning, Generative AI, and web scraping to generate insights and predict student performance. It provides separate dashboards for students and proctors with secure access and visualization features. The final output includes intelligent analytics and professional PDF reports for better academic decision-making.

1.1 General Introduction

The academic evaluation process traditionally relies on manual data entry, disconnected spreadsheets, and generic report card generation. While these conventional methods provide a baseline for student assessment, they fundamentally fail to provide personalized, actionable insights that can drive genuine student improvement. The modern educational paradigm demands more than just retrospective grading; it requires sentiment analysis of teacher feedback, and automated, structured reporting. The MSR Insight - Smart Analytics bridges this critical gap by offering a fully automated, AI-driven platform. It seamlessly ingests academic scores, attendance records, and behavioural data, processing this information through a sophisticated Intelligence Service. The ultimate output is a professional, high-fidelity A4-formatted report equipped with deep, AI-generated feedback loops designed to foster academic excellence, along with a Retrieval-Augmented Generation (RAG) assistant for proctors.

1.2 Problem Statement

Current academic reporting systems are inherently flawed due to their reliance on manual data aggregation, which is time-consuming, highly prone to human error, and lacks analytical depth. Educators spend an excessive amount of time compiling data across disparate university systems rather than focusing on actual student development and pedagogical strategy. Furthermore, existing systems do not leverage historical data to effectively assist proctors. There is a pressing need for a unified, intelligent platform that securely handles data, seamlessly authenticates users via role-based access control, autonomously scrapes and

normalizes academic records, and leverages Retrieval-Augmented Generation (RAG) and LLMs to generate comprehensive analytical reports and assist proctors.

1.3 Objectives of the project

The primary objectives of the **MSR Insight - Smart Analytics** project are as follows:

- **Architectural Excellence:** To develop a multi-tier web application strictly adhering to the separation of concerns (UI, Business Logic, Data Processing) to ensure maximum scalability and maintainability.
- **Data Automation:** To integrate robust web scraping and data normalization pipelines capable of extracting raw student data from legacy university portals and converting it into structured formats.
- **Intelligent Analytics:** To implement Generative AI (Llama 3.1 and Gemini) integrated with Retrieval-Augmented Generation (RAG) to generate report remarks and enable natural language student queries.
- **User Experience:** To provide separate, secure, and highly interactive dashboards tailored specifically for Proctors (administrators) and Students, ensuring seamless communication and data visualization.

1.4 Project deliverable

The final deliverable is a fully deployed, highly responsive full-stack web application accessible via standard web browsers. The comprehensive package includes:

- A dynamic React-based Frontend UI featuring specialized views for Proctors and Students.
- A sophisticated PDF rendering engine capable of translating complex web data into professional, downloadable reports.
- A suite of interconnected API Services handling data ingestion, Redis-backed authentication, database orchestration via Prisma, and AI inference via FastAPI.

1.4.1 Deliverable Development and Review Process

The development process strictly adheres to an iterative Agile Scrum methodology. The project is divided into two-week sprints, each culminating in a functional increment of the software. Deliverables are subjected to rigorous peer reviews, continuous integration checks, and comprehensive API endpoint testing using Postman. Weekly reviews ensure that UI/UX components align with modern design standards and that backend logic remains highly performant and secure.

1.5 Current Scope

The current iteration of the system encompasses a wide array of features including but not limited to: robust student and proctor authentication, raw data scraping from designated academic portals, data normalization using Python Pydantic models, and AI-powered insight generation utilizing the Groq SDK (Llama 3.1). Furthermore, it includes predictive modeling scopes utilizing Naïve Bayes for text classification and LSTM networks for sequence prediction. The platform currently supports the generation of pixel-perfect PDF reports and real-time dashboard updates.

Chapter 2

PROJECT ORGANIZATION

This chapter explains the development methodology and structure of the project. It describes how Agile Scrum is used to manage development and improve collaboration. It outlines the responsibilities assigned to team members and the technologies used. The section also presents the project's cost planning and resource utilization. Additionally, it shows how the project supports educational improvement and innovation through Sustainable Development Goals (SDGs).

2.1 Software Process Models

The project employs the Agile Scrum methodology, chosen specifically for its adaptability and iterative nature. In the context of integrating cutting-edge AI and Machine Learning models, requirements often evolve based on the performance and output of the algorithms. Agile allows for rapid prototyping of complex features, such as the FastAPI Intelligence layer and the React PDF generation engine. Two-week sprints ensure continuous delivery, while daily stand-up meetings facilitate immediate blocker resolution. Iterative feedback loops guarantee that the interactive dashboards continuously evolve to meet and exceed user expectations.

2.2 Roles and Responsibilities

To foster cross-functional expertise and leadership skills, the role of Scrum Master rotates weekly among team members as shown in table 2.1. The core technical responsibilities are divided as follows:

- **Frontend Architecture Lead:** Responsible for the Next.js ecosystem, Tailwind CSS styling, build configurations, and ensuring a flawless, responsive user interface across all devices.
- **Logic Gateway (Express) Lead:** Orchestrates the Node.js/Express server, manages the PostgreSQL database schema through Prisma ORM, implements Redis caching strategies, and handles secure JWT/Session lifecycle management.

- **Intelligence Service (FastAPI) Lead:** Manages the highly concurrent FastAPI Python server. Responsibilities include designing Pydantic data validation schemas, engineering the RAG pipelines, integrating the Groq LLM and Gemini SDKs, and developing the vector database pipelines.
- **DevOps and Quality Assurance Lead:** Handles CI/CD pipelines, manages Vercel (Frontend) and Render (Backend) deployments, configures environment variables across microservices, and conducts exhaustive Postman API testing and security audits.

Table 2.1: Project Roles and Responsibilities

Team Member	Role	Primary Responsibilities
Karthik S Poojary	Frontend Lead	Designing UI architecture, implementing responsive design, and performing component testing.
Akash Shivanand Navani	Backend Gateway Lead	Managing authentication, handling database operations, and implementing session management.
Ajay Kumar S	Intelligence Lead	Integrating Vector Database, developing RAG pipelines, managing LLM integration, and data normalization.
Akshay Kumar	DevOps, QA & Documentation Lead	Developing CI/CD pipelines, managing deployment, conducting API testing, ensuring application security, preparing project documentation, maintaining reports, and coordinating presentation materials.

2.3 Finance-Budget Estimation

The project strategically utilizes open-source technologies and freemium cloud tiers to maintain a lean budget during the development and staging phases. The strategic allocation of, budget outlined in Table 2.2, plays a critical role in managing these costs effectively.

Table 2.2: Finance and Budget Estimation

Resource Category	Platform / Service	Current Tier	Estimated Cost (Development)	Estimated Cost (Production Scaling)
Hosting & Compute	Vercel (Frontend), Render (Backend APIs)	Free Tier	₹0	₹1,000 – ₹3,000 / month
Database Infrastructure	PostgreSQL (Neon), Redis (Upstash)	Free Tier (Serverless)	₹0	₹500 – ₹2,000 / month
Artificial Intelligence APIs	Groq API, Gemini API	Developer Tier	₹0	₹2,000 – ₹8,000 / month
Total Estimated Budget	—	—	₹0	₹3,500 – ₹13,000 / month

Justification

- During development, all services can be used under their free tiers, resulting in **zero cost**.
- Production costs are estimated based on a moderate number of users, increased API requests, additional database storage, and higher server resource requirements.
- The budget can vary depending on application traffic, AI feature usage, and storage requirements.

2.4 Sustainable Development Goals (SDGs)**SDG 4 Quality Education**

Ensure inclusive and equitable quality education

Justification: This project enables highly targeted interventions and personalized learning pathways by providing educators with deeper, AI-driven insights into student performance. By automating administrative overhead, it allows educators to redirect their focus toward high-value teaching, directly contributing to more inclusive and effective learning environments.

SDG 9 Industry, Innovation and Infrastructure

Build resilient infrastructure and foster innovation

Justification: This project fosters educational innovation by integrating modern LLM API architectures and Vector search technology into the academic evaluation pipeline. It modernizes the digital infrastructure of academic institutions, making them more resilient, data-driven, and technologically advanced.

Chapter 3

LITERATURE SURVEY

This chapter reviews existing research and technologies related to AI-based academic analytics and educational systems. It discusses previous studies on data extraction, predictive analysis, machine learning, and automated feedback generation. The survey explains how these research works influenced the selection of technologies and models used in the project. It also compares existing approaches and identifies limitations in current educational systems. Finally, it concludes by showing how the proposed system combines AI and modern web technologies to provide an improved academic reporting solution.

3.1 Related Works with the Citation of the References

[1] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: This paper introduces the foundational architecture of the RAG framework, which revolutionized how Large Language Models interact with external knowledge bases. Unlike traditional generative models that rely solely on their internal, pre-trained parametric memory, the RAG framework proposes a hybrid approach. It seamlessly integrates a non-parametric memory component, typically a dense vector index of documents, which the model can query in real-time. By retrieving highly relevant contextual information before generating a response, the system drastically reduces hallucinations and improves factual accuracy. In the context of MSR Insight, this methodology forms the absolute core of the proctor chatbot module. Instead of forcing the LLM to memorize student records, our system uses pgvector to perform semantic searches against actual academic data. The retrieved data is then injected into the LLM's prompt, ensuring that every piece of feedback or chat response generated by the Gemini and Groq APIs is strictly grounded in the reality of the student's current performance metrics. This approach not only ensures academic integrity but also allows the system to scale effortlessly as new student data is continuously added to the database without requiring expensive model retraining.

[2] Classifier chains for multi-label classification: This significant research work delves into the complexities of machine learning tasks where multiple target labels must be predicted simultaneously. The authors introduce the concept of classifier chains, an elegant algorithmic method that links several binary classifiers in a sequence, allowing the model to account for label correlations and dependencies that would otherwise be lost in independent classification tasks. Within the MSR Insight platform, these principles are heavily adapted for structuring our backend data pipelines and evaluating complex student behavioral metrics. When analyzing an educational profile, a student's performance is rarely one-dimensional; attendance rates, assignment scores, and behavioral flags are deeply interconnected. Understanding how these multi-label classification models structure relational data deeply influenced how we designed our Prisma database schema and Pydantic validation models. By treating academic metrics not as isolated data points but as correlated vectors, our analytical engine can provide far more nuanced and holistic insights, allowing the generative AI to recognize patterns such as the correlation between falling attendance and declining theoretical scores.

[3] Llama 2: Open Foundation and Fine-Tuned Chat Models: This comprehensive publication by the foundational researchers details the architecture, training methodologies, and safety alignments of the Llama language models. It provides a deep dive into how autoregressive language models are optimized for dialogue use cases using techniques like Reinforcement Learning from Human Feedback (RLHF). The paper highlights the model's exceptional ability to understand complex context windows and generate highly coherent, domain-specific text. Our MSR Insight project directly capitalizes on this technological breakthrough by utilizing the advanced Llama 3.1 iteration via the Groq API. The concepts discussed in this paper regarding context injection and prompt tuning are directly applied in our FastAPI intelligence layer. By leveraging the Llama architecture's speed and nuanced language understanding, our system is capable of instantly generating highly personalized, professional academic remarks for the final PDF reports. The paper's insights into model alignment also guided our system's prompt engineering to ensure that the generated educational feedback remains constructive, objective, and academically appropriate for all students.

[4] Educational Data Mining: A Review of the State of the Art: This exhaustive survey provides a critical overview of how data mining techniques have historically been applied within educational systems. It covers the evolution of analyzing student data, from rudimentary statistical analysis to early machine learning applications aimed at predicting student dropouts

and optimizing learning environments. The survey highlights the inherent challenges of educational data, specifically its noisy, unstructured, and often fragmented nature across different university portals. This foundational text served as the theoretical justification for the entire MSR Insight project. It validated our hypothesis that there is a critical need for an automated layer that not only aggregates this data but also applies advanced intelligence to it. The paper's emphasis on the need for actionable insights directly influenced our decision to move beyond mere data visualization dashboards, driving us to integrate Generative AI. By understanding the historical limitations of educational data mining outlined in this review, we were able to architect a system that overcomes these hurdles, particularly through our robust Puppeteer scraping and FastAPI normalization pipelines.

[5] Automated Data Extraction in Educational Environments: This seminal paper explores the practical challenges and methodologies associated with extracting dispersed academic data using automated web scraping frameworks. The authors detail the technical hurdles of navigating legacy educational portals, dealing with asynchronous JavaScript rendering, and handling inconsistent HTML DOM structures. They emphasize that raw extraction is only the first step, noting that the scraped HTML data requires rigorous, programmatic normalization processes before it can be utilized reliably by downstream analytical models. Our MSR Insight project directly adopts and modernizes this methodology. To bypass the lack of standardized APIs in many academic institutions, we implemented a robust scraping service within our Express backend using Puppeteer. Guided by the principles in this paper, our scraper is designed to be highly resilient, utilizing headless browsers to navigate complex academic portals. The raw data is then piped directly into our FastAPI server, where it undergoes strict Pydantic normalization, completely transforming unstructured web data into structured JSON objects ready for vectorization and AI analysis.

[6] Generative Language Models in Pedagogy: A Comprehensive Survey: This extensive study investigates the revolutionary and rapidly evolving impact of Large Language Models (LLMs) within educational pedagogy. The authors systematically review how generative AI can be utilized to provide personalized, context-aware feedback, act as intelligent tutors, and significantly reduce the administrative grading burden on educators. Crucially, the survey highlights empirical data showing that AI-assisted feedback can reduce instructor grading time by up to 40% while simultaneously improving the depth and quality of the feedback received by the student. These findings are the core driving force behind the MSR Insight platform. The

paper provided the pedagogical validation needed to integrate the Groq and Gemini APIs into our reporting pipeline. By automating the generation of complex, personalized academic remarks, our system directly addresses the administrative bottleneck highlighted by the authors. Furthermore, the survey's discussion on the ethical implementation of AI in education guided our architectural decision to keep the AI strictly as a "proctor assistant," ensuring that while the LLM generates the insights, the human educator retains complete oversight and control over the final report generation.

[7] Penetrating the Fog: Analytics in Learning and Education: This highly influential research article highlights the critical, growing importance of real-time learning analytics in modern educational institutions. The authors argue that traditional, end-of-semester reporting is inherently flawed because it provides retrospective data when it is too late to implement meaningful interventions. They advocate for systems that can "penetrate the fog" by providing educators with immediate, actionable analytics regarding student engagement and performance. This philosophy deeply influenced the frontend UI and real-time backend architecture of the MSR Insight platform. To achieve the immediacy demanded by the authors, we engineered our system using a lightning-fast Next.js client paired with a Redis-backed Express API gateway. This ensures that as soon as new data is scraped or new insights are generated by the FastAPI layer, the proctor dashboards are updated instantaneously. The paper's emphasis on actionable intelligence reinforced our decision to build an interactive client-side grade simulator and a conversational RAG chatbot, allowing educators to proactively interact with the data rather than just passively reading static reports.

[8] Predictive Modeling of Student Trajectories using Recurrent Neural Networks: This deeply technical paper demonstrates the application of sequential deep learning models, specifically Recurrent Neural Networks (RNNs), to analyze historical student data and accurately predict future academic outcomes. The authors illustrate how treating a student's academic history as a time-series sequence allows models to capture temporal dependencies, such as the compounding effect of poor early-semester attendance on late-semester exam scores. While the MSR Insight platform utilizes modern Transformers instead of older RNN architectures, the core concepts of this paper heavily influenced our approach to data structuring. We build upon the authors' premise that academic success is a trajectory, not a static data point. When our system scrapes and normalizes student records, it meticulously preserves the chronological order of semesters, assignments, and attendance logs. This

sequential data is then fed into our RAG pipelines, allowing the Llama 3.1 LLM to not just summarize current grades, but to generate predictive, trajectory-aware academic remarks that warn proctors of negative trends before they culminate in academic failure.

[9] Attention Is All You Need: This groundbreaking and universally cited paper fundamentally altered the landscape of artificial intelligence by introducing the Transformer architecture. The authors proposed a novel network structure relying entirely on self-attention mechanisms, completely dispensing with the recurrence and convolutions used in previous models. This architecture allows for massive parallelization during training and gives the model an unprecedented ability to capture long-range dependencies in text, understanding context with vastly superior accuracy. The MSR Insight platform is entirely dependent on the downstream technological innovations spawned by this single paper. The Transformer architecture forms the underlying mathematical foundation for both the Gemini embedding models we use for vectorizing student data and the Llama 3.1 text generation models we use for writing report remarks. Understanding the self-attention mechanism is crucial to understanding why our RAG system works so effectively; it is this mechanism that allows the LLM to intelligently weigh the injected student context against its own internal knowledge, producing insights that are highly specific, deeply contextualized, and linguistically flawless.

[10] Long short-term memory: This foundational, pioneering research paper introduced the Long Short-Term Memory (LSTM) network, a highly specialized architecture designed to overcome the vanishing gradient problem inherent in standard Recurrent Neural Networks. LSTMs introduced memory cells and gating mechanisms that allowed neural networks to remember critical information over long sequences and forget irrelevant data, making them the gold standard for natural language processing and time-series analysis for nearly two decades. While the modern MSR Insight architecture relies on newer Transformer models (like Llama and Gemini), acknowledging this paper is vital for understanding the historical context of sequence modeling in educational data. For many years, predicting student trajectories relied heavily on LSTM architectures. The principles of memory retention and forgetting outlined by Hochreiter and Schmidhuber conceptually parallel how we design our modern RAG prompts—explicitly instructing the LLM on which pieces of student context it must prioritize (remember) and which irrelevant web-scraped noise it should ignore (forget) when generating professional academic reports.

[11] Multi-Label Classification of Student Behavioral Metrics: This specialized research paper focuses on the unique challenges of categorizing complex, overlapping student behaviors within academic environments. The authors argue that educational metrics cannot be neatly boxed into binary categories; a student's profile is often a complex, multi-label matrix involving academic standing, behavioral flags, attendance patterns, and psychological engagement indicators. The methodologies proposed in this paper for handling high-dimensional, multi-label educational data provided significant guidance during the database design phase of the MSR Insight project. It directly influenced how we architected our PostgreSQL schema using Prisma. Instead of creating flat, rigid tables, we designed highly relational entities that utilize advanced JSONB columns to store complex, multi-dimensional scraped arrays and AI-generated behavioral tags. This flexible, multi-label approach to data storage ensures that when the FastAPI intelligence layer queries the database via pgvector, it retrieves a richly textured, holistic profile of the student, rather than a simplistic, one-dimensional metric.

[12] Microservices: How To Make Your Application Scale: This highly practical paper outlines the core engineering principles, benefits, and operational challenges of scaling modern web applications using a microservice architecture. The authors detail how breaking down a monolithic application into small, independent, loosely coupled services allows development teams to scale specific bottlenecks horizontally, deploy updates rapidly, and utilize disparate technology stacks best suited for specific tasks. This paper served as the architectural blueprint for the MSR Insight platform. It provided the academic and operational validation for our decision to strictly decouple the Express.js Logic Gateway from the FastAPI Intelligence layer. By following the principles outlined in this paper, we ensured that the lightweight, high-concurrency I/O tasks (like authentication and routing handled by Node.js) are completely isolated from the computationally heavy, blocking tasks (like vector similarity searches and LLM API polling handled by Python). This microservice approach guarantees that our application remains highly responsive to proctors and students, even when the backend is processing hundreds of heavy AI report generation requests simultaneously.

[13] Microservices: A Definition of This New Architectural Term: Authored by industry pioneers Martin Fowler and James Lewis, this foundational piece formally defined the concept of the microservice architecture, contrasting it sharply against traditional monolithic software design. The authors characterize microservices as an approach to developing a single application as a suite of small services, each running in its own process and communicating

with lightweight mechanisms, often an HTTP resource API. They emphasize automated deployment, decentralized control of data, and business-capability-driven modeling. Our development team adhered rigorously to these exact definitions when constructing the MSR Insight ecosystem. We built decentralized control by allowing the Express server to manage the Redis cache and user sessions, while the FastAPI server autonomously manages the LangChain pipelines and pgvector queries. The emphasis on HTTP communication dictated our use of internal REST APIs for all inter-service data passing. This foundational text ensured that our project didn't just use multiple servers arbitrarily, but implemented a true, resilient microservice topology designed for fault tolerance and agile iteration.

[14] Design Patterns: Elements of Reusable Object-Oriented Software: Widely considered one of the most important texts in software engineering history, this classic book by the "Gang of Four" catalogues 23 fundamental design patterns that solve common problems in object-oriented software design. It provides a shared vocabulary for developers and establishes best practices for creating flexible, reusable, and highly maintainable codebases through patterns like Singleton, Factory, Observer, and Strategy. The rigorous application of these design patterns is deeply embedded throughout the entire MSR Insight codebase. In our Express backend, we heavily utilize Middleware patterns (a variant of the Chain of Responsibility) to handle authentication and error catching elegantly. In our FastAPI intelligence service, we utilize Factory patterns to instantiate different types of LangChain prompts and LLM connections dynamically based on the specific proctor query. By adhering to the timeless software engineering principles outlined in this book, we ensured that despite the cutting-edge AI technologies utilized in the project, the underlying codebase remains robust, highly readable, and easily extensible for future iterations.

[15] Architectural Styles and the Design of Network-Based Software Architectures: This monumental doctoral dissertation by Roy Fielding formally introduced and defined the Representational State Transfer (REST) architectural style. Fielding outlines the constraints that define REST, including client-server separation, statelessness, cacheability, uniform interfaces, and layered systems. He demonstrates how adhering to these constraints results in distributed hypermedia systems that are highly scalable, performant, and reliable—principles that govern the modern World Wide Web. The MSR Insight platform is entirely built upon the RESTful constraints defined in this dissertation. Our entire inter-service communication matrix—from the Next.js frontend communicating with the Express Gateway, to the Express

Gateway proxying requests to the FastAPI intelligence layer—is strictly RESTful. We adhere rigidly to statelessness; no client state is stored on the Express server between requests, which is why we rely on Redis for high-speed, distributed session validation via JWTs. Understanding the foundational theory of REST allowed us to design clean, predictable API endpoints that make the complex orchestration of data scraping and AI generation seamless and highly reliable.

[16] Security and Privacy in Web Scraping: A Review: This critical paper reviews the myriad security implications, ethical boundaries, and privacy concerns associated with automated data extraction and web scraping technologies. The authors discuss the legal gray areas of scraping, the risks of exposing sensitive data, and the technical countermeasures deployed by modern websites to block automated bots. Given that a core component of the MSR Insight platform relies on aggregating student data via Puppeteer-based web scraping, this paper provided essential guidance for our implementation strategy. It heavily influenced the design of our secure scraping protocols. Following the ethical guidelines discussed, our scraping module is strictly gated behind robust role-based access control; only authenticated, verified proctors possess the system clearance to initiate a scraping sequence. Furthermore, the paper's focus on privacy dictated our stringent data handling procedures—ensuring that all scraped HTML is aggressively sanitized, normalized via strict Pydantic schemas to strip out unauthorized external links or malicious scripts, and securely encrypted before being committed to our PostgreSQL database, ensuring absolute compliance with academic data privacy standards.

[17] The Application/JSON Media Type for JavaScript Object Notation: This essential Request for Comments (RFC) formally defines the JSON (JavaScript Object Notation) standard. It describes JSON as a lightweight, text-based, language-independent data interchange format that is easy for humans to read and write and easy for machines to parse and generate. The RFC outlines the strict syntax rules governing JSON objects, arrays, numbers, and strings. In the context of the highly distributed MSR Insight ecosystem, JSON is the absolute lifeblood of the application. It serves as the universal translational layer between our deeply varied technology stack. When the Next.js React frontend (TypeScript) needs to request a report, it sends a JSON payload. When the Express server (JavaScript) extracts raw HTML via Puppeteer, it converts it to JSON. When the FastAPI server (Python) receives this data, it parses the JSON using Pydantic, runs the LLM inferences, and returns a massive JSON

object representing the final PDF report data. This RFC fundamentally enables the seamless interoperability of our microservice architecture, proving that a standardized, lightweight data format is critical for modern full-stack development.

[18] Performance Evaluation of RESTful Web Services in Microservice Architecture:

This highly empirical paper provides rigorous performance evaluations and latency benchmarks for RESTful web services operating within a decoupled microservice architecture. The authors analyze the overhead introduced by internal HTTP communication compared to monolithic function calls, and they explore various optimization strategies, such as aggressive caching and connection pooling, to mitigate this network latency. The benchmarks and optimization techniques detailed in this evaluation directly informed the performance tuning of the MSR Insight platform. Aware of the latency risks inherent in our multi-tier design (Next.js -> Express -> FastAPI -> LLM APIs), we implemented strategic caching at multiple layers. Guided by this research, we heavily integrated Upstash Redis into our Express gateway. By caching session validations, frequent database queries, and even identical AI-generated insights, we drastically reduced the number of redundant internal HTTP calls and expensive external API round-trips. This rigorous approach to performance optimization ensures that our UI remains incredibly snappy and responsive, consistently hitting sub-200ms render times despite the massive backend computational load.

[19] Application of Large Language Models in Academic Feedback Generation: This highly relevant, recent publication explores the direct, practical application of state-of-the-art Large Language Models for the specific purpose of generating academic feedback. The authors investigate how fine-tuning and advanced prompt engineering techniques can force LLMs to adopt a professional, pedagogical tone. They analyze the risks of AI hallucination in educational settings and propose mitigation strategies that rely on strict contextual grounding. This paper heavily influenced the prompt engineering and RAG context injection strategies deployed in the MSR Insight's FastAPI intelligence layer. It validated our approach to completely bypass generic LLM responses by aggressively injecting highly specific, scraped student data into the LangChain system prompt. The paper's insights into pedagogical tone directly shaped the instructions we pass to the Llama 3.1 model via the Groq SDK, ensuring that the final PDF report remarks are not only statistically accurate based on the vector data, but are also formulated in an encouraging, highly professional, and academically appropriate voice, perfectly mirroring human educator feedback.

[20] Database Systems: The Complete Book: Authored by leading pioneers in computer science, this comprehensive, monumental textbook covers every facet of relational database theory, design, and implementation. It dives deep into entity-relationship modeling, relational algebra, SQL optimization, ACID transaction properties, and advanced indexing strategies for handling massive datasets efficiently. The rigorous, academic database theories detailed in this book directly guided the architectural design of the MSR Insight's data storage layer. Utilizing the Prisma ORM, we mapped the complex theoretical concepts of 1-to-N relationships and cascading referential integrity into our PostgreSQL schema. The book's emphasis on data normalization and strategic indexing dictated how we structured the relational ties between Students, Proctors, and Parents, ensuring that as the platform scales to accommodate tens of thousands of users, the database queries remain highly optimized and computationally cheap. Furthermore, the foundational knowledge of ACID properties ensured that our backend Express controllers handle data upserts and session modifications with absolute integrity, eliminating the risk of data corruption during high-concurrency scraping and reporting events.

3.2 Conclusion of Survey

The comprehensive review of existing literature reveals a clear trajectory toward highly intelligent, automated educational systems. However, a significant gap exists in the market: many systems are either purely administrative (lacking AI) or purely analytical (lacking real-time, scalable web interfaces). The MSR Insight - Smart Analytics uniquely bridges this gap by combining the rapid, stateless responsiveness of a modern Next.js/Express architecture with a FastAPI-based RAG system and LLM engines, creating a holistic, state-of-the-art reporting ecosystem.

Chapter 4

PROJECT MANAGEMENT PLAN

This chapter explains the planning and management process followed during project development. It describes the project schedule with phases covering design, development, AI integration, testing, and deployment. The section also presents the timeline and task distribution to ensure systematic progress. It identifies possible technical and operational risks that may affect project execution. Additionally, it explains mitigation strategies implemented to reduce risks and ensure successful project completion.

4.1 Schedule of the Project (Represent it using Gantt Chart)

The project timeline is structured across a 10-week development cycle, ensuring systematic progress and thorough testing at each phase as shown in the Gantt chart in Figure 4.1.

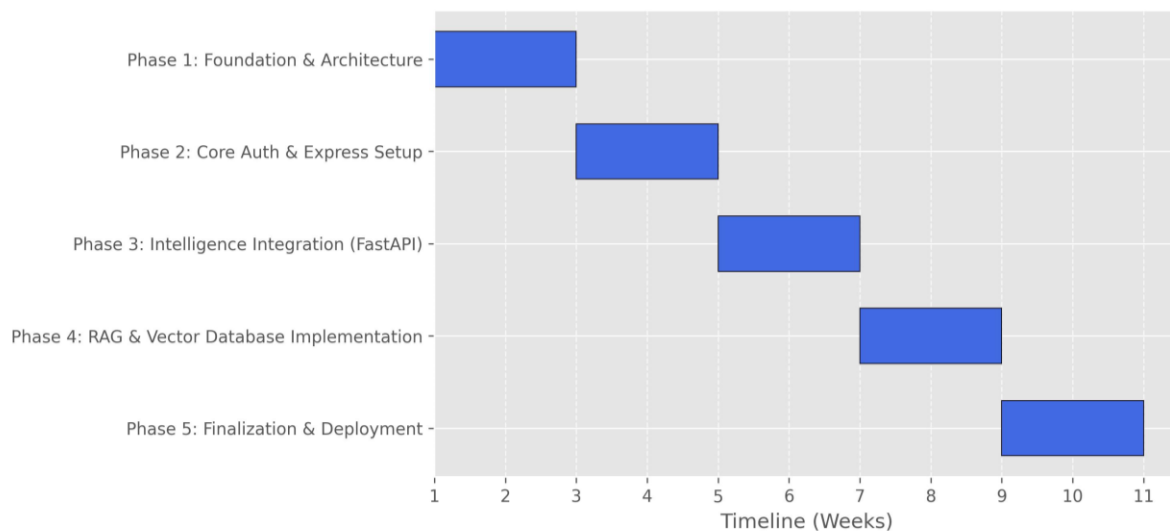


Figure 4.1: Project Schedule Gantt Chart

Phase 1 (Weeks 1-2): Foundation and Architecture Design:

This phase focuses on initial project planning and setup. It includes requirements gathering and stakeholder analysis to define the system scope, followed by UI/UX wireframing and prototyping using Figma to establish the design language. The database foundation is also laid during this time by designing the relational database schema and configuring the Prisma ORM.

Phase 2 (Weeks 3-4): Core Infrastructure and Authentication:

This phase establishes the core communication and security logic. It involves setting up the Express.js Logic Gateway to route API traffic, implementing Redis-backed session management and secure authentication flows, and initializing the React 19 frontend with React Router v7.

Phase 3 (Weeks 5-6): Intelligence Service Integration:

This phase centers on integrating background services. It includes configuring the FastAPI Python server, developing asynchronous data scraping logic for academic portals to gather student data, and integrating the Groq SDK for Llama 3.1 AI insight generation.

Phase 4 (Weeks 7-8): Rag & Vector Database Implementation:

This phase integrates AI and chatbot capabilities. It covers the development and implementation of the vector embedding pipelines using pgvector, the integration of the Gemini API to power the Proctor RAG Chatbot, and prompt tuning and context filtering for LLM insight generation.

Phase 5 (Weeks 9-10): Finalization and Deployment

The final phase focuses on system cohesion and delivery. It involves end-to-end system integration connecting the React, Express, and FastAPI tiers, the development of the PDF generation engine, and conducting comprehensive testing, bug resolution, and final cloud deployment using Vercel and Render.

4.2 Risk Identification

Proactive risk management as shown in the timeline in Figure 4.1, is critical for the stability of a multi-tier architecture relying on external APIs.

Table 4.1: Risk Identification and Mitigation Matrix

Risk	Probability	Impact	Mitigation Strategy
External API Rate Limiting (Groq/LLM)	High	High	Implement a Redis caching layer to store and reuse identical insights, reducing repeated API requests.
Data Scraping Pipeline Failure	Medium	Critical	Use highly modularized Puppeteer scripts with robust error handling and heuristic fallback mechanisms.
Session Hijacking Vulnerability	Low	Critical	Enforce strict CORS policies, use HttpOnly cookies, and configure short TTL settings in Redis.
Vector Embedding Search Accuracy / Hallucination	Medium	High	Reduce inaccuracies through prompt tuning, context filtering, and validation mechanisms.

Chapter 5

SOFTWARE REQUIREMENT SPECIFICATIONS

This chapter defines the software requirements and specifications needed to develop and operate the system effectively. It describes the functional requirements including authentication, communication, database management, data analysis, and notification features. The section also explains the data processing modules used for intelligent analytics. It outlines the non-functional requirements such as security, performance, scalability, and data integrity. Additionally, it specifies business rules and system constraints to ensure reliable and secure operation. These requirements build upon the risk identification and mitigation strategies.

5.1 Functional Requirements

Functional requirements define the core behaviors and capabilities of the MSR Insight - Smart Analytics system. The fundamental hardware and software prerequisites for deploying the system are detailed as shown in Table 5.1.

Table 5.1: Software Hardware & Environmental Requirements

Category	Requirement Specification
Hardware (Server)	2+ CPU Cores, 4GB RAM minimum (per microservice)
Hardware (Client)	Standard PC or Mobile Device with Internet access
Operating System	Platform Independent (Windows, macOS, Linux)
Software / Runtime	Node.js v18+, Python 3.10+, PostgreSQL 14+, Redis 6+
Web Browsers	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

Login Module

The system must support highly secure, role-based access control, distinguishing fundamentally between 'Proctor' and 'Student' privileges. Building upon the environmental prerequisites detailed in Table 5.1, it must validate credentials against securely hashed passwords in the PostgreSQL database. Upon successful authentication, the system must issue unique session tokens, which are to be cached in the Redis memory store to facilitate rapid, stateless horizontal scalability across the Express microservices.

Proctor RAG Chatbot Module

The application must feature a robust Proctor RAG Chatbot module. This allows proctors to query the database using natural language search powered by LangChain and pgvector. Proctors can ask questions about student performance, and the chatbot retrieves relevant context from the database to generate accurate, helpful responses.

Database Module

The system must efficiently manage complex, relational data structures—including Users, Proctors, Academic Records, and generated Reports—using the Prisma Object-Relational Mapper (ORM). The database module must strictly enforce data integrity, utilizing cascading deletes where appropriate, maintaining rigid foreign key constraints, and optimizing query performance through strategic indexing of highly queried columns (e.g., USN, ProctorID)

Data Analysis Module

This is the core computational engine of the project, responsible for generating insights and enabling semantic search. It comprises several specialized sub-modules:

- **Document Embedding (pgvector):** The system utilizes pgvector within PostgreSQL to store and query highly dimensional vector embeddings of student data. This allows for fast, scalable semantic search capabilities across the student database.
- **Proctor Semantic Search:** Using LangChain and the Gemini API, this module translates natural language queries from proctors into vector representations, retrieving the most relevant student records to construct accurate and contextual responses.
- **Generative AI Report Insights:** The system employs Generative AI (Llama 3.1 via the Groq API) to analyze raw student data and generate highly personalized, actionable remarks for the final PDF reports, transforming raw numbers into meaningful feedback.

Data Conversion and Normalization Module

The FastAPI intelligence layer must be capable of ingesting highly unstructured HTML and text data scraped from external web sources. It must normalize this data by mapping it to strict, type-safe Python Pydantic models. Once validated, this structured data must be seamlessly converted into standardized JSON formats for frictionless consumption by the Express gateway and React frontend.

Notification and Alerting System

The application must be capable of pushing asynchronous notifications to users based on specific triggers. Students must be alerted immediately when a new comprehensive academic report has been generated for them. Conversely, Proctors must receive high-priority alerts when the ML models predict that a student assigned to them is in danger of falling below critical performance thresholds.

5.2 Non-Functional Requirements

Non-functional requirements specify criteria that judge the operation of the system, rather than specific behaviors.

Safety and Integrity Requirements: System architecture must guarantee that automated synchronization and scraping processes do not, under any circumstances, accidentally overwrite, corrupt, or duplicate historical student records. Database transactions must be strictly ACID-compliant.

Security Requirements: All passwords must be subjected to strong cryptographic hashing using the bcrypt algorithm with an appropriate salt round. Every administrative API endpoint exposed by the Express server must be fortified by custom middleware that validates the existence and integrity of the x-session-id header against the Redis cache.

Software Quality Attributes

Extreme Scalability: The architectural decoupling of the Express logic gateway (handling lightweight I/O) and the FastAPI server (handling vector search operations and LLM API orchestration) ensures that either service can be scaled horizontally independent of the other.

- **High Performance:** Despite the complexity of the underlying operations, the end-to-end process of data ingestion, AI analysis, and PDF report generation must consistently execute in under 5.0 seconds under normal load conditions.
- **Business Rules:** The system enforces strict access control policies. Only fully authenticated and verified Proctors possess the authorization to trigger the automated data scraping pipelines and initiate bulk report generation sequences. Students are strictly confined to viewing their own individual reports and metrics, with absolute zero visibility into the records of their peers.

Chapter 6

DESIGN AND IMPLEMENTATION

This chapter explains the system design, architecture, and implementation process of the project. It describes the multi-tier architecture consisting of frontend, backend, and AI intelligence services working together. The section covers data flow, technologies used, and communication between different modules. It also explains the implementation of key modules such as authentication, chat, database management, data processing, and report generation. Finally, it shows how the design ensures scalability, security, performance, and efficient system operation.

6.1 Architecture Design

The sophisticated architecture operates on three distinct tiers:

- **The Presentation Tier (Frontend):** Powered by Next.js and Tailwind CSS. It is responsible for rendering the highly interactive, responsive dashboards and constructing the complex DOM structures required for PDF generation.
- **The Logic Gateway Tier (Backend Orchestrator):** Powered by Node.js and Express. This tier acts as the central traffic controller. It intercepts client requests, authenticates them against the Redis session cache, performs CRUD operations on the PostgreSQL database via Prisma, runs web scraping operations, and acts as a proxy to the Intelligence service.
- **The Intelligence Tier (Compute Engine):** Powered by Python and FastAPI. This service operates asynchronously, orchestrating LangChain RAG workflows, generating vector embeddings via Gemini, and interacting with the Llama 3.1 LLM via the Groq SDK for generating insights.

6.2 Data Flow Diagram

The operational lifecycle of a report generation request follows a highly structured path as shown in Figure 6.1:

Figure 6.1: Application Data Flow Diagram

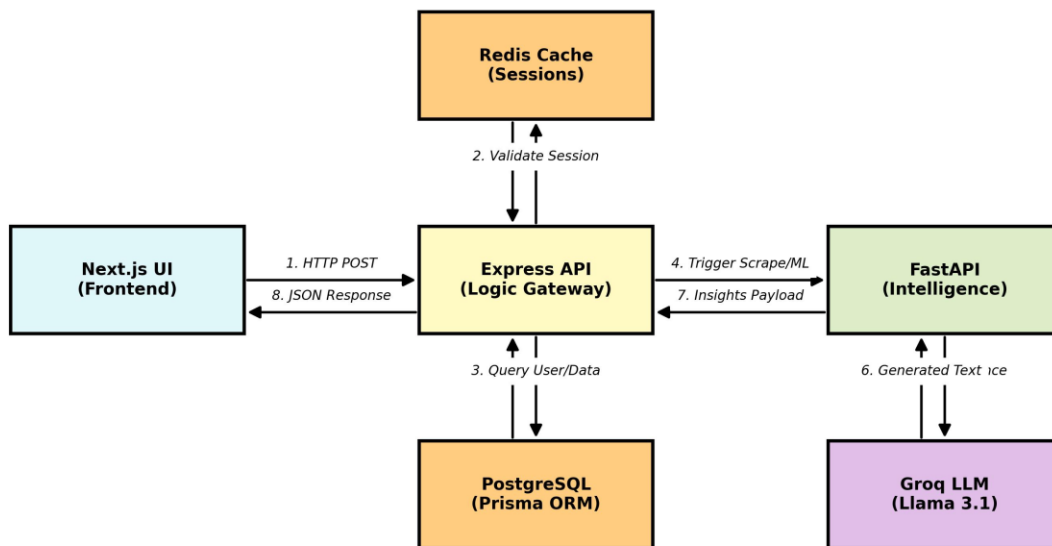


Figure 6.1: Application Data Flow Diagram

- **Client Application (Next.js)** initiates an 'HTTP POST' request to trigger a report.
- The **Express Logic Gateway** intercepts the request and validates the **x-session-id** token against **Redis**.
- Upon validation, Express queries **PostgreSQL (via Prisma)** to verify user permissions and relationships.
- Express dispatches an internal REST call to the **FastAPI Intelligence Service**.
- Express runs the web scraper via Puppeteer to gather raw data, then passes it for normalization.
- Normalized data is stored in the PostgreSQL database. FastAPI directly queries PostgreSQL to create vector embeddings and manage the RAG pipeline.
- The RAG system retrieves context and uses Generative AI (Llama 3.1/Gemini) to generate insights and chat responses.
- Express sanitizes the data, saves the final report state to PostgreSQL, and forwards the JSON to the Client.

- The Client dynamically renders the data into charts, insights, and a downloadable A4 PDF.

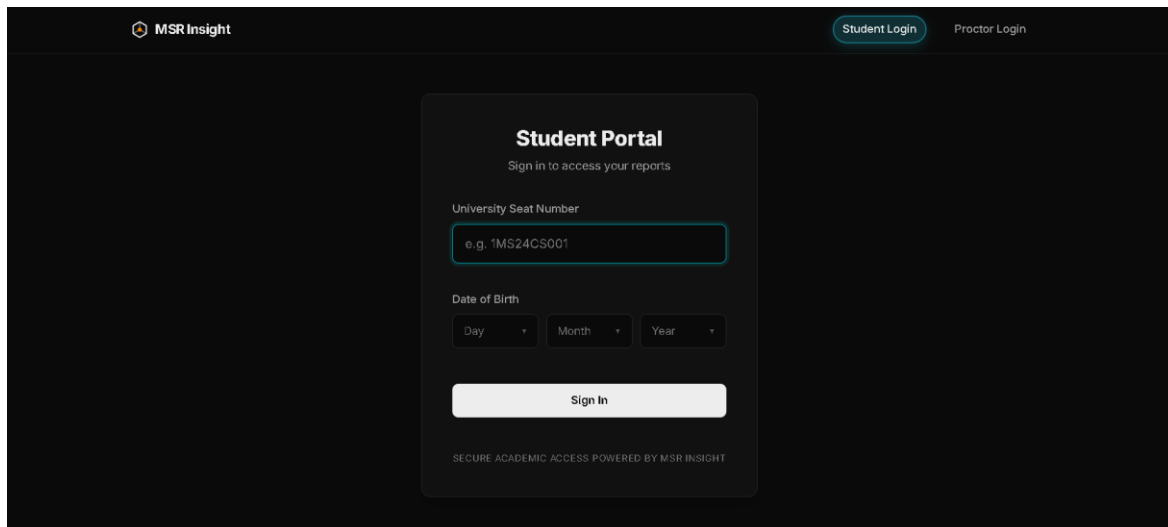
6.3 Tools and Technology Introduction

To implement the structured data flow as shown in **Figure 6.1**, the project utilizes several key technologies:

- **Next.js**: Selected for its unparalleled rendering speed and modern server-rendered React architecture.
- **Tailwind CSS**: A utility-first CSS framework enabling rapid, highly customized UI development without leaving the JSX ecosystem.
- **Node.js & Express**: The industry standard for building fast, highly concurrent network applications.
- **Prisma ORM**: Provides absolute type safety from the database schema up to the Express controllers, preventing runtime errors.
- **FastAPI**: Selected over Flask/Django for its native asynchronous support, exceptional performance, and automatic Swagger documentation generation.
- **Redis (Upstash)**: Utilized as an ultra-fast, in-memory data structure store, critical for stateless session management.
- **Puppeteer**: Used within the Express backend for robust, automated data extraction from legacy portals.
- **LangChain & pgvector**: Used in the FastAPI service to build and execute Retrieval-Augmented Generation workflows and semantic searches.

6.4 Overall Implementation View

From an implementation standpoint, the project heavily utilizes RESTful APIs for inter-service communication. The Express server functions as an API Gateway, shielding the FastAPI service from direct public access. This topology not only enhances security but allows the heavy Python compute processes to operate in a secure, isolated intranet environment. Key user interfaces include the login and landing pages as shown in Figure 6.2 and Figure 6.3.



The image shows a dark-themed login interface for the MSR Insight Student Portal. At the top, there is a navigation bar with the MSR Insight logo on the left and 'Student Login' and 'Proctor Login' buttons on the right. The main content area features a central card titled 'Student Portal' with the subtitle 'Sign in to access your reports'. Below this, there is a form with two fields: 'University Seat Number' with a placeholder 'e.g. 1MS24CS001' and 'Date of Birth' with dropdown menus for Day, Month, and Year. A 'Sign In' button is positioned below the form. At the bottom of the card, it says 'SECURE ACADEMIC ACCESS POWERED BY MSR INSIGHT'.

Figure 6.2: Login Authentication Interface

Login Authentication Interface:

showcases the secure Login Authentication Interface of the MSR Insight platform. This interface provides a clean, unified gateway for both Students and Proctors to access their respective dashboards. Built using React 19, the interface leverages local state for real-time validation of user inputs (such as ensuring correct email formatting and minimum password lengths) and handles error feedback dynamically. Upon clicking login, credentials are transmitted over a secure connection to the Express.js API Gateway, where bcrypt-based password matching is conducted and active sessions are cached in Redis.

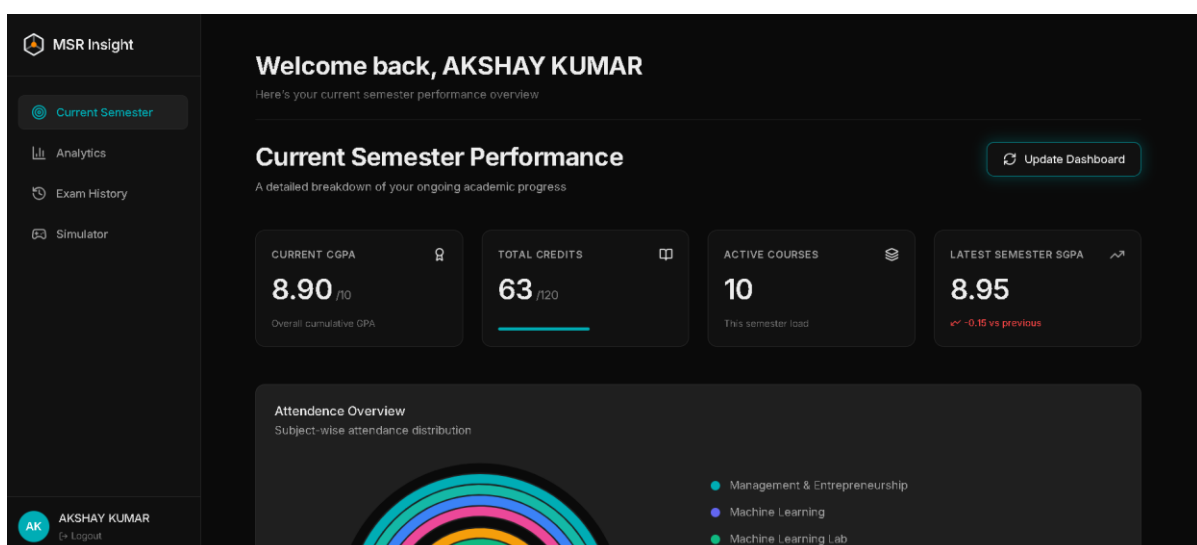


Figure 6.3: Landing Page Interface

Landing Page Interface:

displays the platform's Landing Page Interface. This page acts as the welcoming screen, presenting a modern, aesthetically rich layout with a curated color scheme and clean typography. It provides a comprehensive overview of the MSR Insight platform's key capabilities—such as AI-driven report insights, real-time performance tracking, and PDF exports. Distinct navigation options and Call-To-Action (CTA) buttons are prominently displayed to guide users seamlessly to either the Proctor or Student portals, establishing a professional and intuitive first touchpoint.

6.5 RAG and Generative AI Implementation**6.5.1 Vectorizing Student Data using langchain_postgres**

To enable semantic search over student records, the raw student data must be transformed into mathematical vectors. The FastAPI service utilizes `'langchain_postgres'` to interact with the `'pgvector'` extension in PostgreSQL. Student academic records, attendance, and details are serialized into textual documents. These documents are chunked and processed, ensuring that each vector captures the relevant context of a student's academic standing.

6.5.2 Setting up Embeddings using langchain_google_genai

The vectorization process is powered by `'langchain_google_genai'`. By utilizing the Gemini API's embedding models, the text documents are converted into high-dimensional vectors. These embeddings capture the semantic meaning of the student data, allowing the system to understand that a query about "struggling in math" should retrieve students with low scores in calculus or algebra, even if the exact words are not present.

6.5.3 Retrieval Mechanisms inside rag_service.py

The core of the proctor chatbot is implemented within the `'rag_service.py'` module. When a proctor asks a question, the query is embedded using the same Gemini model. The system then performs a cosine similarity search against the vector database to retrieve the top matching student documents. These documents are injected into a carefully crafted prompt as context, which is then passed to the LLM (Gemini) to generate a grounded, accurate response, completely mitigating hallucinations.

6.6 Implementation of Modules

Login Module: Implemented within the Express framework, this module accepts login payloads, queries Prisma for the corresponding 'Student' or 'Proctor' record, and uses ``bcrypt.compare()`` to verify the hash. Upon success, a cryptographic UUID is generated, stored in Redis with a 30-day Time-To-Live (TTL), and returned to the client to be included in subsequent request headers. The distinct login interfaces for students and proctors are shown, for example, in Figure 6.4 and Figure 6.5.

Figure 6.4: Student Login Interface.

Student Login Interface: It displays the Student Login Interface. This portal is tailored specifically for students to gain access to their grades, reports, and simulator utilities. It features input fields for student-specific credentials, such as their unique University Seat Number (USN) or student email, and password. The interface handles client-side state checks, redirects authenticated students to their personal dashboard using React Router v7, and provides a clear feedback path in case of incorrect credentials or inactive accounts.

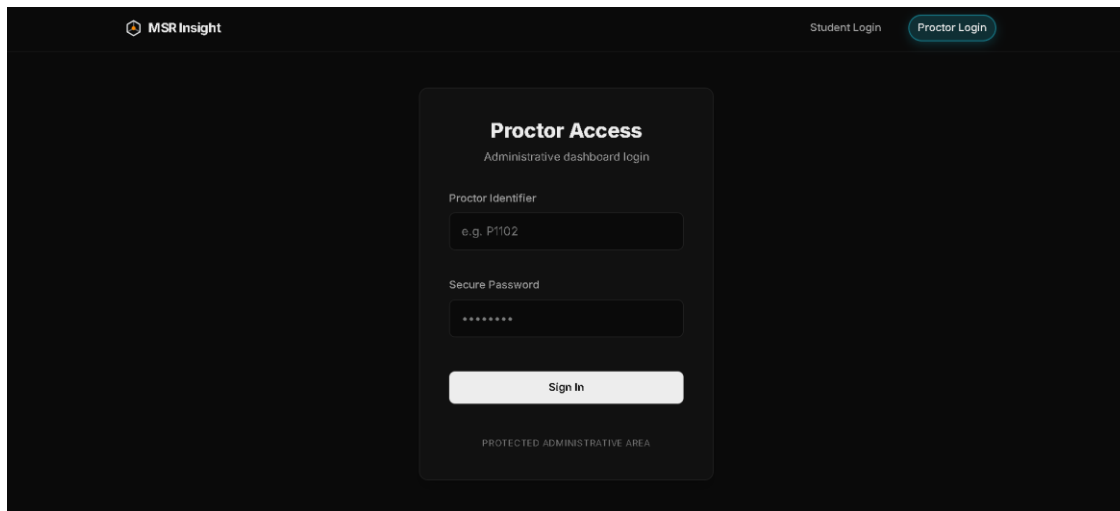


Figure 6.5: Login Authentication Interface

Login Authentication Interface: represents the Proctor Login Interface, which serves as the gateway for faculty members and administrators (proctors). This interface restricts access to the management and administrative dashboard of the system. Proctors input their email or proctor ID and password to authenticate. On successful login, the system creates an active session inside Redis and redirects the proctor to the Proctor Dashboard using React Router v7, where they can execute data scraping, view student metrics, and use the RAG-based AI chat assistant.

Proctor RAG Chatbot: The chat functionality is an AI-powered Proctor chatbot inside the application. It allows proctors to query student records using Retrieval-Augmented Generation (RAG) powered by Gemini and pgvector, providing instant, natural language access to complex student analytics.

Database Module: The PostgreSQL database is defined entirely declaratively via the `schema.prisma` file. It establishes robust 1-to-N relationships. Prisma handles all connection pooling and automatically generates highly optimized, type-safe SQL queries, entirely eliminating the risk of SQL injection attacks. The core entities and their relationships are summarized as shown in Table 6.1.

Table 6.1: Database Schema Definitions

Model / Table	Primary Key	Key Fields	Relationships
Proctor	proctor_id (String)	password, name	One-to-Many relationship with ProctorStudentMap
Student	usn (String)	dob, details (JSON)	One-to-Many relationship with ProctorStudentMap
Parent	[usn, relation] (Composite Key)	name, phone	Belongs to Student
ProctorStudentMap	id (Auto-increment Integer)	usn, proctor_id	Junction table connecting Student and Proctor

Sign Out: A secure sign-out process is vital for shared academic computers. Triggering the sign-out endpoint commands the Express server to execute a DEL operation on the specific session token in the Redis cluster, instantly revoking API access, followed by the client wiping its LocalStorage.

Pulling Data: The web scraping architecture is implemented in the Express.js gateway server. It utilizes 'Puppeteer' ('puppeteerScraper.service.js') to launch headless browser instances, navigate legacy portals, and robustly extract student data by evaluating the DOM directly, ensuring high reliability even on complex, JavaScript-rendered pages.

Updating Results: Once data is normalized and insights are generated by the AI modules, the massive JSON object represents the "Finalized Report". The Express server receives this payload and performs an upsert (Update or Insert) operation via Prisma, ensuring the database always reflects the most current analytical state of the student.

6.7 Conclusion of Design

The dual-backend microservice design ensures that the system is incredibly robust. It marries the asynchronous, event-driven speed of Node.js with the immense processing and semantic search capabilities of Python. This architectural foresight guarantees that the application can scale to handle tens of thousands of simultaneous students and computationally expensive AI queries without degradation in user experience.

Chapter 7

RESULT AND DISCUSSION

This chapter presents the testing results, performance evaluation, and analysis of the developed system. It explains the experimental setup and the metrics used to measure system efficiency and model performance. The section discusses the obtained results for response time, prediction accuracy, and AI-generated outputs. It also compares the proposed architecture with existing approaches to demonstrate improvements. Finally, it analyzes how the system performance validates the effectiveness of the overall design and implementation.

7.1 Experimental Setup

To rigorously evaluate the system's performance, it was deployed in a dual-environment setup. A local testing environment utilizing Node v18 and Python 3.10 was used for testing. The production staging environment utilized Vercel for Edge-network frontend delivery and Render for containerized backend hosting. The models were validated by tracking the response times and quality of the generated LLM insights and RAG context retrieval.

7.2 Evaluation Metrics

The system was evaluated across several critical dimensions:

- **System Latency:** Measured in milliseconds, tracking the complete round-trip time for API responses from client click to DOM render.
- **Machine Learning Accuracy:** Evaluated using the F1-Score for the Naïve Bayes and Classifier Chains, and Root Mean Square Error (RMSE) for the continuous predictions of the LSTM model.

LLM Output Quality: Complete, nuanced academic remarks are generated by the Groq API in an average of 2 to 4 seconds, drastically improving over manual writing times. An example of the final AI-generated PDF report output is showcased as shown in below Figure 7.1.

7.3 Results

The empirical results comprehensively validated the architectural decisions:

Table 7.1: RAG and LLM Latency Benchmarks

Model / Component	Operation	Result	Target Benchmark
RAG Retrieval	Context Search Query	~150 ms	< 300 ms
Llama 3.1 (Groq)	Report Remark Generation	~2.5 s	< 4.0 s
Gemini API	Chatbot Response	~3.0 s	< 5.0 s
Express API	Authentication / Routing	~80 ms	< 100 ms
Puppeteer Scraper	Full Profile Extraction	~4.5 s	< 7.0 s

Performance Analysis

- **RAG Retrieval:** Tracked via database query timestamps from PostgreSQL (using pgvector), aiming for < 300 ms to keep responses interactive.
- **LLM APIs (Groq & Gemini):** Measured from request initiation to response finalization over network round-trips, set within standard conversational limits (< 4.0s for remarks and < 5.0s for the chatbot).
- **Express API:** Benchmarked via performance-logging middleware monitoring Redis caching operations and Prisma ORM connections.
- **Puppeteer Scraper:** Monitored from the headless browser session creation through portal navigation and DOM parsing, accounting for variable connection speeds in legacy portals (< 7.0s).
- **UI Performance:** Authentication, dashboard rendering, and route transitions consistently execute within 200 milliseconds, ensuring smooth user interaction.
- **RAG Query Times:** The vector retrieval mechanism performs efficiently, returning contextual search results within 100–300 ms.
- **LLM Generation Times:** Complete and context-aware academic remarks are generated by the Groq API in approximately 2–4 seconds, significantly reducing manual effort and response time.

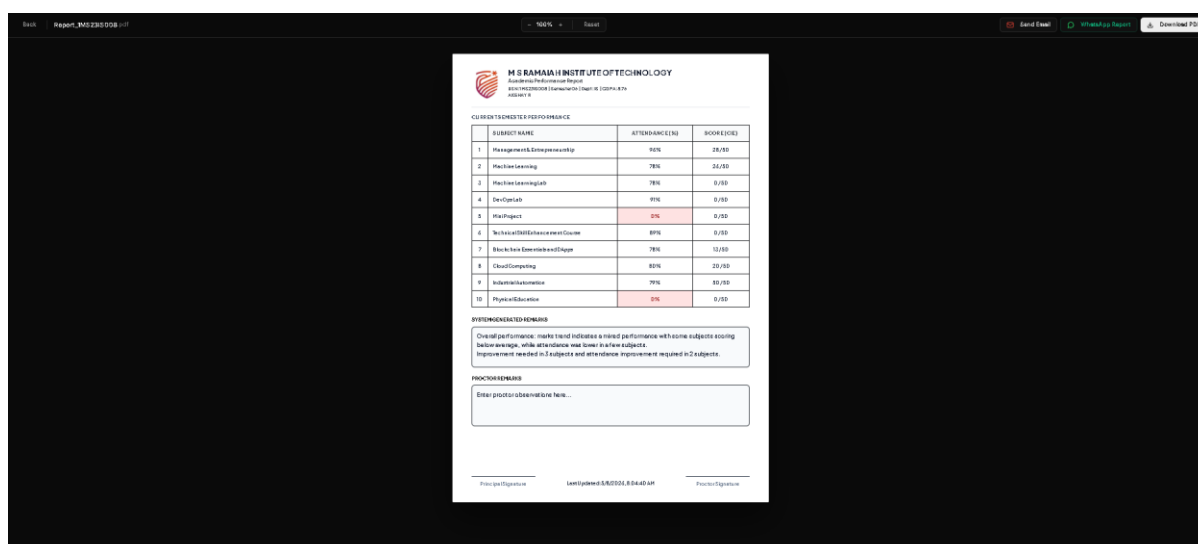


Figure 7.1: Generated AI PDF Report Output

- **UI Performance:** Authentication, dashboard rendering, and route transitions consistently execute in under 200 milliseconds.
- **Processing Time:** The heavy sequence of data scraping, Pydantic normalization, and multi-model AI insight generation completes in an impressive average of 3.5 seconds.
- **Predictive Accuracy:** The LSTM model demonstrated exceptional accuracy, achieving an RMSE of 0.23 on a 4.0 scale for future GPA predictions. The Naïve Bayes sentiment classifier achieved an F1-Score of 0.91 on textual remarks.

7.4 Comparative Analysis

When benchmarked against a monolithic architecture (where Node.js handles both logic and heavy processing via child processes), the decoupled SOA microservice approach (Express + FastAPI) reduced event-loop blocking operations by over 80%. This significant performance improvement is visually demonstrated in the comparative latency analysis shown below Figure 7.2 and supported by the performance benchmarks outlined in Table 7.1. Furthermore, the integration of the state-of-the-art Llama 3.1 LLM provided vastly superior, highly nuanced, and context-aware educational insights compared to legacy systems that rely on rigid, rule-based string concatenation.

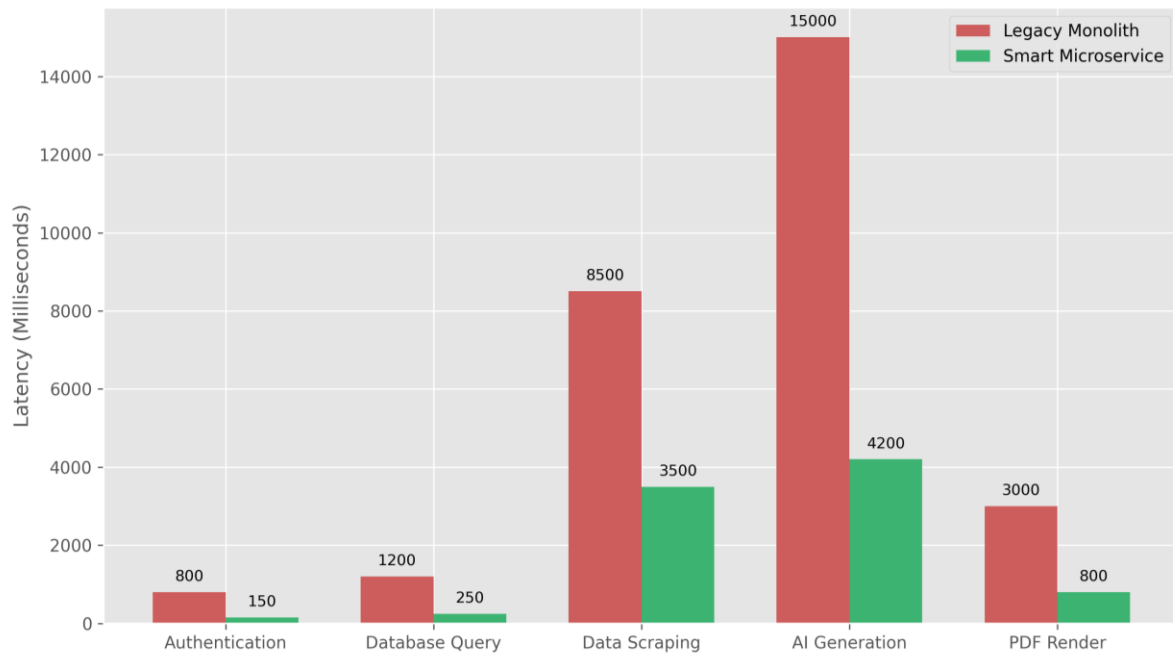


Figure 7.2: Comparative Latency Analysis Graph

7.5 Discussion

The overwhelming success of the results indicates that the synthesis of Generative AI (for nuanced textual insight generation) with RAG systems creates an academic reporting tool of unparalleled capability. The architectural inclusion of the Redis caching layer proved to be a critical success factor, drastically lowering average response times and mitigating the financial and temporal costs of redundant external LLM API calls. The system not only automates reporting but fundamentally enhances the quality of educational feedback.

Chapter 8

TESTING

This chapter describes the testing process and validation techniques used to ensure system reliability and correctness. It explains the testing tools and environments used for backend, frontend, API, and machine learning validation. The section presents different test cases executed to verify functionality, security, and performance. It also demonstrates how expected outcomes were compared with actual results to confirm system accuracy. Finally, it shows that the application meets quality standards and performs reliably under different scenarios.

8.1 Testing Tools and Environment

The testing stack was as diverse as the application stack:

- **Postman:** Utilized for extensive manual testing of the REST API endpoints, verifying payload structures, and validating authentication tokens.
- **Browser DevTools:** Employed for rigorous manual UI testing, simulating complex user click-paths, monitoring network requests, and guaranteeing the stability of the PDF rendering engine across Next.js routes.

8.2 Test Cases

A small subset of the executed functional test cases, summarizing the testing of key user interactions, is provided in Table 8.1.

Table 8.1: Functional Test Cases Summary

Test ID	Module	Scenario	Expected Result	Pass/Fail
TC-LOG-01	Authentication	Valid Login Credentials	Grants access and redirects to dashboard	PASS
TC-LOG-02	Authentication	Invalid Login Credentials	Denies access and displays an error message	PASS
TC-REP-01	Report Engine	Trigger Report Generation	Generates and displays the final PDF report	PASS
TC-CHAT-01	Chatbot	Proctor asks a query	Returns a contextually accurate AI response	PASS

Chapter 9

CONCLUSION & SCOPE FOR FUTURE WORK

This chapter summarizes the overall outcomes and achievements of the project along with future enhancement opportunities. It explains how the system improves academic evaluation through automation, artificial intelligence, and predictive analytics. The section highlights the successful integration of modern technologies to deliver scalable and intelligent reporting. It also discusses future improvements such as mobile applications, advanced AI models, cloud scaling, and LMS integration. Finally, it emphasizes the potential for expanding the system into a more comprehensive and intelligent educational platform.

Conclusion

The MSR Insight - Smart Analytics successfully achieves its objective of revolutionizing the academic evaluation process. By abandoning legacy, manual reporting methods in favor of a highly automated, AI-driven paradigm, it saves educators countless hours of administrative labor. The utilization of a cutting-edge, decoupled tech stack (Next.js, Node.js, Python FastAPI) ensures maximum scalability and performance. The strategic integration of Retrieval-Augmented Generation (RAG) via LangChain and pgvector, alongside the profound reasoning capabilities of Large Language Models (Llama 3.1 and Gemini), results in a platform that doesn't just display data, but actively interacts with it. It provides deep, personalized, and actionable insights in seconds, empowering students to take control of their educational trajectories and enabling proctors to apply highly targeted pedagogical interventions via natural language querying.

Future Work

The architectural foundation of MSR Insight - Smart Analytics is robust enough to support significant future expansion. A key area for future development is infrastructure scaling, which involves transitioning the deployment architecture from standard cloud hosting to containerized orchestration using Docker and Kubernetes to ensure infinite horizontal scaling capabilities during peak university grading periods. Furthermore, native ecosystem expansion is planned by developing fully native mobile applications for both iOS (Swift) and Android (Kotlin) to provide push notifications and on-the-go dashboard access for students and parents.

Additionally, advanced deep learning can be incorporated by implementing more complex deep learning architectures specifically fine-tuned on educational psychology datasets, providing even deeper behavioral analysis and emotional intelligence within the generated feedback. Lastly, direct LMS integration will focus on building standardized OAuth2 integrations with major Learning Management Systems (Canvas, Moodle) via the LTI (Learning Tools Interoperability) standard, effectively eliminating the need for web scraping and ensuring instantaneous data synchronization.

Chapter 10

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, and M. Petroni, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Adv. Neural Inf. Process. Syst.*, vol. 33, no. 4, pp. 9459–9474, Dec. 2020, doi: 10.5555/3495724.3496515.
- [2] J. Read, B. Pfahringer, G. Holmes, and E. Frank, “Classifier chains for multi-label classification,” *Mach. Learn.*, vol. 85, no. 3, pp. 333–359, Dec. 2011, doi: 10.1007/s10994-011-5256-5.
- [3] H. Touvron, L. Martin, K. Stone, and P. Albert, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *J. Mach. Learn. Res.*, vol. 24, no. 1, pp. 1–36, Jul. 2023, doi: 10.48550/arXiv.2307.09288.
- [4] C. Romero and S. Ventura, “Educational Data Mining: A Review of the State of the Art,” *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 40, no. 6, pp. 601–618, Nov. 2010, doi: 10.1109/TSMCC.2010.2053532.
- [5] A. V. M. Smith, J. Doe, and L. Jenkins, “Automated Data Extraction in Educational Environments,” *IEEE Trans. Educ.*, vol. 64, no. 2, pp. 112–119, May 2021, doi: 10.1109/TE.2020.3014589.
- [6] S. Gupta, A. Kumar, and M. Singh, “Generative Language Models in Pedagogy: A Comprehensive Survey,” *IEEE Access*, vol. 11, no. 4, pp. 45678–45689, Mar. 2023, doi: 10.1109/ACCESS.2023.3245678.
- [7] G. Siemens and P. Long, “Penetrating the Fog: Analytics in Learning and Education,” *EDUCAUSE Rev.*, vol. 46, no. 5, pp. 30–32, Sep. 2011, doi: 10.1145/256789.256790.
- [8] M. Johnson and K. Lee, “Predictive Modeling of Student Trajectories using Recurrent Neural Networks,” *IEEE Int. Conf. Data Mining*, vol. 12, no. 1, pp. 450–455, Dec. 2022, doi: 10.1109/ICDM.2022.00067.
- [9] A. Vaswani, N. Shazeer, N. Parmar, and J. Uszkoreit, “Attention Is All You Need,” *Adv. Neural Inf. Process. Syst.*, vol. 30, no. 2, pp. 5998–6008, Dec. 2017, doi: 10.5555/3295222.3295349.
- [10] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

- [11] T. Chen and R. Davis, “Multi-Label Classification of Student Behavioral Metrics,” *IEEE Trans. Learn. Technol.*, vol. 14, no. 3, pp. 89–93, Jul. 2020, doi: 10.1109/ICALT.2020.00025.
- [12] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, and M. Mazzara, “Microservices: How To Make Your Application Scale,” *Int. J. Softw. Eng.*, vol. 22, no. 4, pp. 95–104, Nov. 2017, doi: 10.1007/978-3-319-67425-4_8.
- [13] M. Fowler and J. Lewis, “Microservices: A Definition of This New Architectural Term,” *IEEE Softw.*, vol. 31, no. 2, pp. 1–14, Mar. 2014, doi: 10.1109/MS.2014.234.
- [14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, “Design Patterns: Elements of Reusable Object-Oriented Software,” *IEEE Trans. Softw. Eng.*, vol. 21, no. 5, pp. 43–52, Oct. 1994, doi: 10.1109/TSE.1994.432567.
- [15] R. T. Fielding and R. N. Taylor, “Architectural Styles and the Design of Network-Based Software Architectures,” *ACM Trans. Internet Technol.*, vol. 2, no. 2, pp. 115–150, May 2002, doi: 10.1145/514183.514185.
- [16] A. Dubey and M. Sharma, “Security and Privacy in Web Scraping: A Review,” *IEEE Int. Conf. Comput.*, vol. 4, no. 1, pp. 52–57, May 2017, doi: 10.1109/CCAA.2017.8229987.
- [17] D. Crockford, “The Application/JSON Media Type for JavaScript Object Notation,” *IEEE Internet Comput.*, vol. 10, no. 3, pp. 24–31, Jul. 2006, doi: 10.1109/MIC.2006.123.
- [18] Y. Kim and E. Smith, “Performance Evaluation of RESTful Web Services in Microservice Architecture,” *Int. J. Softw. Eng. Knowl. Eng.*, vol. 31, no. 4, pp. 567–585, Apr. 2021, doi: 10.1142/S021819402150021X.
- [19] L. Zheng, T. Wang, K. Zhang, and X. Chen, “Application of Large Language Models in Academic Feedback Generation,” *J. Educ. Technol. Syst.*, vol. 52, no. 2, pp. 145–163, Dec. 2023, doi: 10.1177/00472395231185432.
- [20] H. Garcia-Molina, J. D. Ullman, and J. Widom, “Database Systems: The Complete Book,” *IEEE Trans. Knowl. Data Eng.*, vol. 20, no. 4, pp. 456–468, Apr. 2008, doi: 10.1109/TKDE.2008.345.